Statik Kod Analizi ve Hata Tespit Sistemi

Necmettin Erbakan Üniversitesi

CodeSentinel Team | BIL204 | 2026

Proje Özeti

Amacımız

C dilinde yazılmış kaynak kodları çalıştırmadan analiz eden bir statik kod analiz sistemi geliştirmek. Potansiyel hataları erken aşamada tespit ederek yazılım geliştirme sürecinde güvenilirliği artırmak.

Teknoloji

C++ programlama dili kullanılarak nesne yönelimli programlama prensiplerine uygun şekilde geliştirilen masaüstü uygulama.

Neden Statik Analiz?

Güvenlik

Bellek yönetimi problemleri ve güvenlik açıklarını çalışma zamanından önce tespit etmek

Performans

Yüksek performans ve donanım seviyesine yakın kontrol sağlama

Erken Tespit

Derleme aşamasında tespit edilemeyen hataları çalışma zamanından önce bulmak

C programlama dili, esnek ve düşük seviyeli yapısı nedeniyle geliştiricilerin hata yapma olasılığını artırır. Bu sistem, bu riskleri minimize eder.

Hedef Kullanıcılar

Yazılım Geliştiriciler

C dili ile sistem, gömülü veya düşük seviyeli yazılım geliştiren kullanıcılar

Sistem Programcıları

Kod kalitesi ve hata tespiti konusunda hızlı geri bildirim almak isteyen kullanıcılar

Öğrenciler

Syntax, kod kalitesi ve statik analiz mantığını eğitim amaçlı inceleyen kullanıcılar

Sistem Mimarisi

Dosya Yönetimi

Yükleme, doğrulama ve validasyon işlemleri

Ön İşleme

Yorum ayıklama ve tokenization

Analiz Motorları

Syntax, kural ve metrik analizleri

Sonuçlar ve Raporlama

Seviye belirleme ve dışa aktarma

Temel İşlevler

01

Dosya Yükleme ve Doğrulama

C kaynak kodu dosyasını yükleme ve uzantı, okunabilirlik kontrolü

02

Yorum Ayıklama

Yorum satırlarını ayıklama ve satır numarası bütünlüğünü koruma

03

Kod Ayrıştırma

Tokenlara ayırma ve AST (Abstract Syntax Tree) oluşturma

04

Syntax Hata Tespiti

Syntax hatalarının tespiti ve satır bazında raporlanması

05

Kural Analizi

Kod kalitesi kurallarının çalıştırılması ve hata listesi oluşturma

06

Metrik Hesaplama

Kod metriklerinin hesaplanması ve sonuçların görüntülenmesi

07

Raporlama

Bulgu satırına gitme, filtreleme ve kod parçasını vurgulama

Kapsam ve Sınırlar

Dosya Formatı

Sistem yalnızca .c uzantılı C kaynak kodlarını analiz edecektir

Analiz Türü

Analiz işlemi tamamen statik analiz ile yapılacak, kod çalıştırılmayacaktır

Analiz Kapsamı

Temel syntax ve kod kalitesi analizi kapsama dahil olacaktır; tam ölçekli derleyici/IDE entegrasyonu kapsam dışıdır

Platform

Sistem masaüstü ortamda çalışacak şekilde tasarlanacaktır

Odak Noktası

İleri düzey otomatik düzeltme mekanizmaları yerine hata tespiti, raporlama ve kullanıcıya yol gösterme işlevlerine odaklanılacaktır

İş Paketleri

İP 1: Kod İşleme

C kodunu temizler. Kütüphaneleri koda gömer. C kodunu parçalara ayırır. Tokenleri C'nin kurallarına göre birleştirerek anlamlı bir Ağaç oluşturur. Syntax hatalarını yakalar. Sembol tablosu oluşturur ve semantik hataları ayıklar.

İP 2: Bellek Yönetimi

Pointer ve bellek yönetimiyle ilgilenir. Pointer ile ilgili hataları burada yakalar.

İP 3: Sistem Optimizasyonu

Bu sistemin çalışmasını sağlar. Kişiler arasındaki ilişkiden sorumludur. Optimizasyon yapılacaksa çoklu çekirdek ve bellek yönetimi burada yapılır.

İP 4: Kullanıcı Arayüzü

Son kullanıcının ilgilendiği kısımdır. Terminalde ya da GUI'de nasıl görüneceğine, hataların görselleştirilmesine, rapor üretimiyle, sistemin çalışıp çalışmadığıyla ilgili son testlerin yapılmasıyla ilgilenir.

Kullanım Durumu (Use Case) Modeli

Sistemin tüm fonksiyonel gereksinimleri kullanım durumları ile modellenmiştir. Diyagramda kullanıcı ve sistem etkileşimi gösterilmiştir.

UC-01: C Kodu Yükle

Kullanıcının .c uzantılı dosyayı sisteme yüklemesi

UC-02: Analiz Başlat

Analiz sürecinin tüm alt adımlarını tetikleyen orkestrasyon noktası

UC-03: Dosya Doğrula

Uzantı, boşluk ve okunabilirlik kontrolü

UC-04: Kod Ayrıştır

Tokenization ve AST oluşturma

UC-05: Yorum Ayıkla

Tek ve çok satırlı yorumların analiz dışı bırakılması

UC-06: Syntax Hataları Tespit Et

Eksik noktalı virgül, eşleşmeyen parantez vb. hataların bulunması

Use Case

UC-07: Kural Analizi Yap

Kullanılmayan değişken, magic number gibi kod kalitesi kurallarının denetlenmesi

UC-08: Metrik Hesapla

LOC, fonksiyon sayısı, struct sayısı gibi metriklerin hesaplanması

UC-09: Sonuçları Görüntüle

Analiz sonuçlarının kategorize edilerek listelenmesi

UC-10: Uyarı Seviyesi Belirle

Bulgulara Kritik, Uyarı veya Bilgi seviyesi atanması

UC-11: Rapor Oluştur

Analiz sonuçlarının yapılandırılmış rapor olarak derlenmesi

UC-12: Analiz Sonucunu Kaydet

Sonuçların TXT, JSON veya HTML formatında diske kaydedilmesi

UC-13: Kod Satırına Git

Seçilen bulgudaki satıra otomatik navigasyon

UC-14: Hata Türüne Göre Filtrele

Sonuçların önem seviyesine göre filtrelenmesi

UC-15: Kod Parçasını Vurgula

İlgili kod bloğunun görsel olarak vurgulanması

Sınıf Diyagramı - Sistem Mimarisi

Uygulama Çekirdeği

- **Application:** Sistemin giriş noktası ve yaşam döngüsünü yönetir
- **AnalysisEngine:** Analiz sürecinin merkezi birimidir.

Dosya Yönetimi

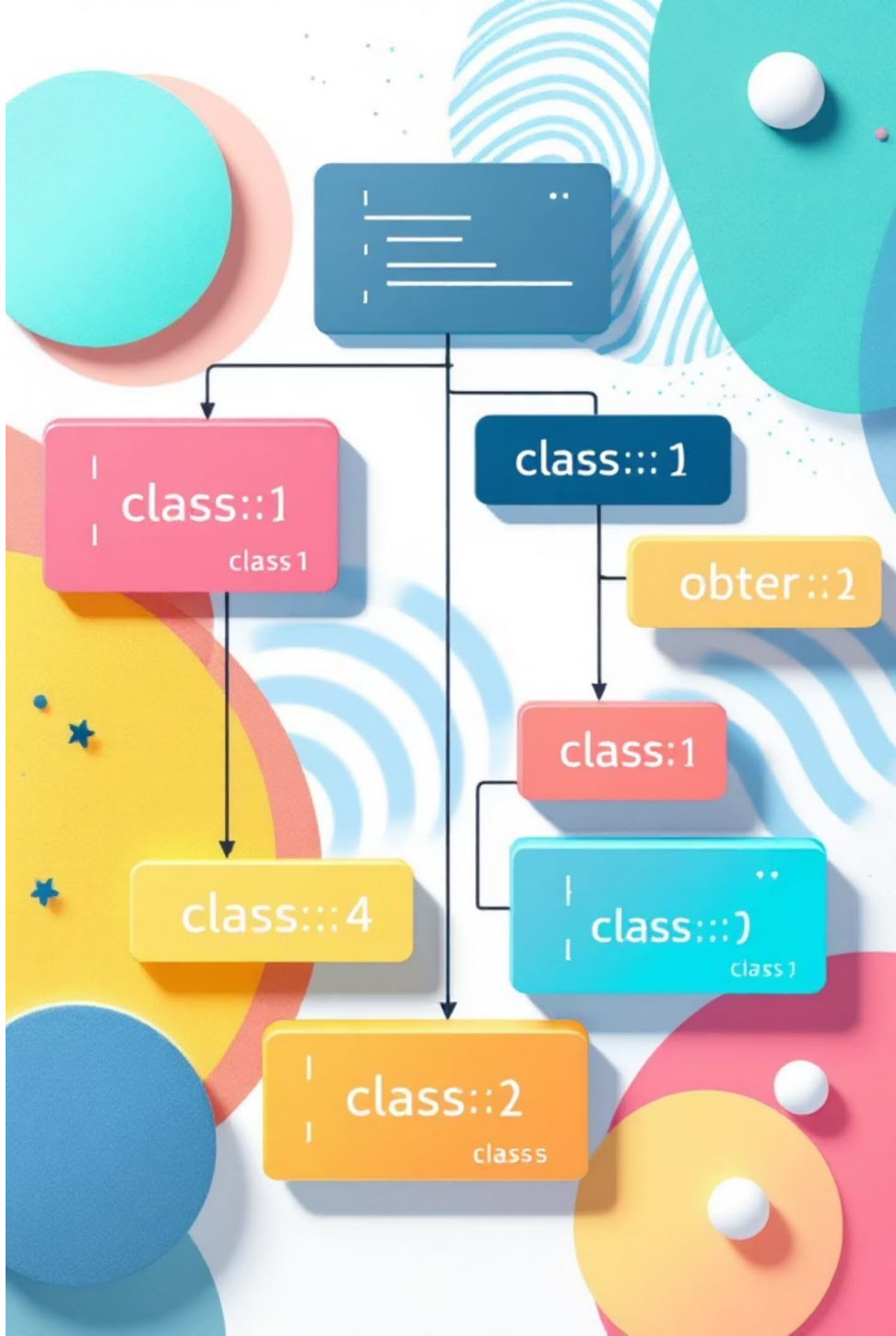
- **SourceFile:** C kaynak kod dosyasını temsil eder
- **FileLoader:** Diskten dosya okuma işlemini gerçekleştirir
- **FileValidator:** Dosya uygunluğunu kontrol eder

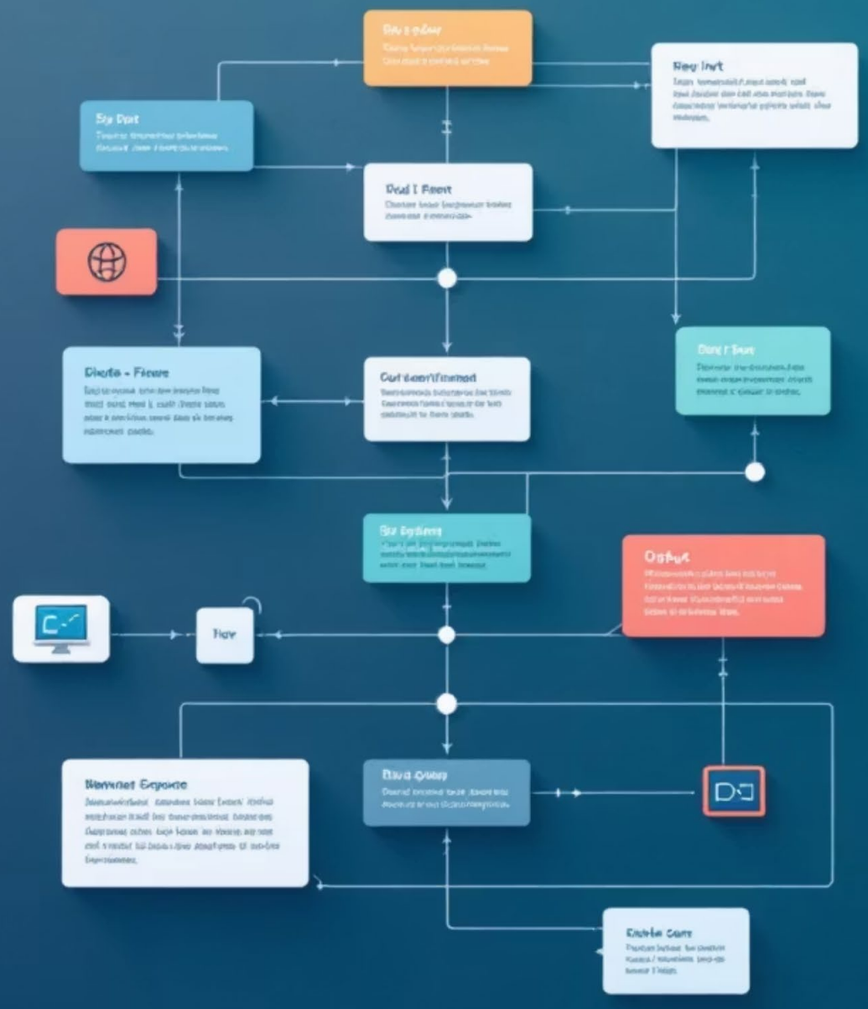
Ön İşleme ve Ayrıştırma

- **Preprocessor:** Yorumları ayıklayarak temizlenmiş kod üretir
- **Lexer:** Kaynak kodu tokenlara ayırır
- **Parser:** Token dizisinden AST oluşturur

Analiz Motorları

- **SyntaxAnalyzer:** Syntax hatalarını tespit eder
- **RuleEngine:** IRule nesnelerini yönetir
- **MetricCalculator:** Kod metriklerini hesaplar





Durum ve Aktivite Diyagramları

1

Analiz Süreci Durumu

Boşta → Dosya Yükleniyor → Dosya Doğrulanıyor → Ön İşleme → Kod Ayrıştırma → Syntax Analizi → Kural Analizi → Metrik Hesaplama → Uyarı Seviyesi → Sonuçlar Hazır

2

Dosya Doğrulama Akışı

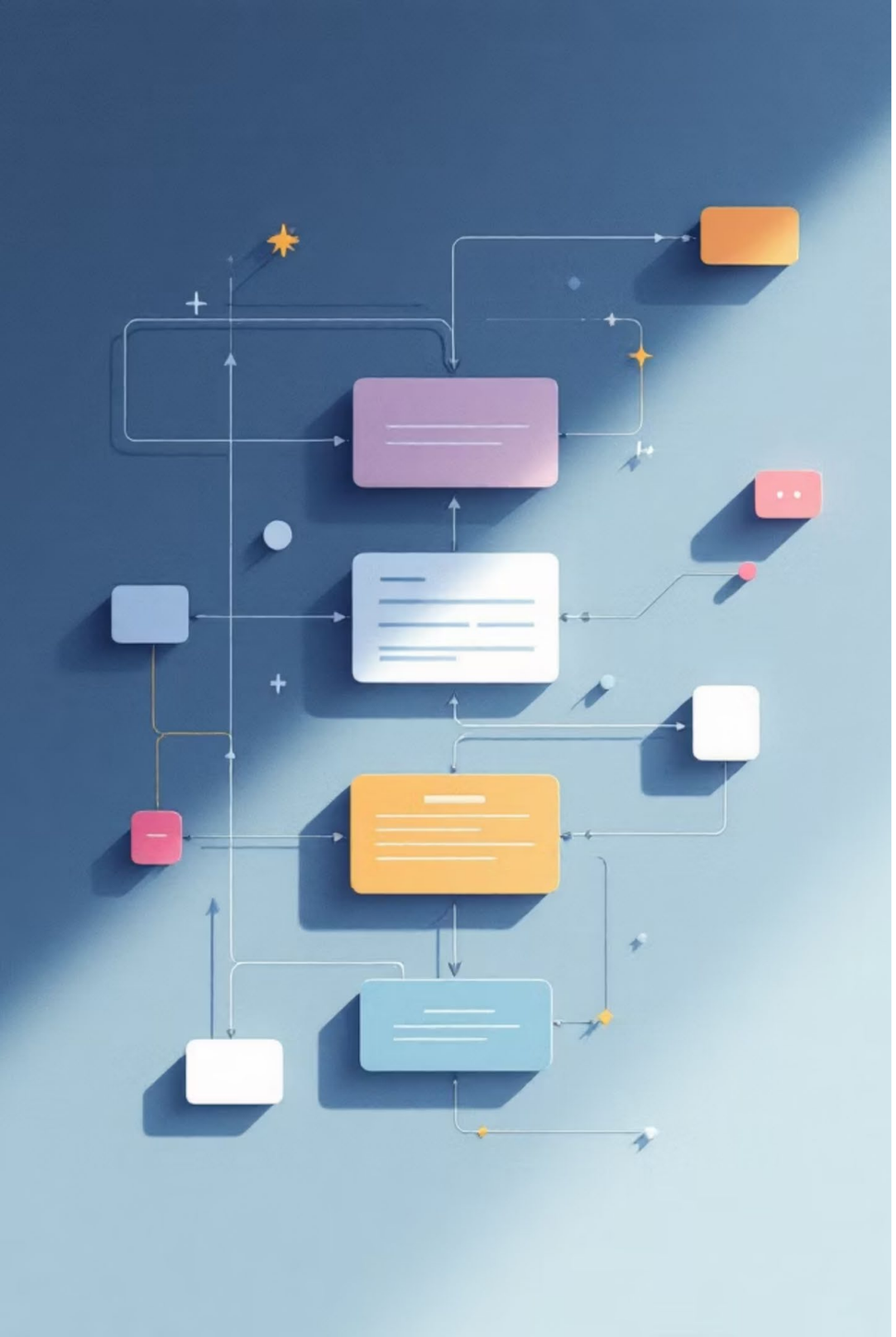
Uzantı Kontrolü → İçerik Kontrolü → Boşluk Kontrolü → Doğrulama Başarılı/Başarısız

3

Rapor Oluşturma

Rapor Bekleniyor → Rapor Oluşturuluyor → Format Seçimi → Dosyaya Yazılıyor → Rapor Tamamlandı/Hatası

Paralel yapı ile syntax hata tespiti, kural analizi ve metrik hesaplama eş zamanlı yürütülür, analiz performansını artırır.



Sıra Diyagramları - Nesne Etkileşimleri

Standart Analiz Akışı

Kullanıcı analiz komutu verir → Dosya işleyici yorumları ayıklar → Lexer tokenization yapar → Parser AST oluşturur → Kurallar çalıştırılır → Bulgular toplanır

Hata Yönetimi

Dosya doğrulama otomatik başlar → Format kontrolü yapılır → Hata yakalanırsa süreç kesilir → Kullanıcıya uyarı mesajı gösterilir

Syntax Hatası Tespiti

Parser AST oluşturma sırasında hata tespit eder → Önem seviyesi ve renk kodu atanır → Kullanıcı sonuç listesinden hataya tıklar → Editörde ilgili satıra kaydırma → Kod bloğu görsel olarak vurgulanır

Raporlama ve Kayıt

Kullanıcı "Rapor Oluştur" tıklar → Analiz sonuçları ve metrikler derlenir → HTML/JSON/TXT formatına dönüştürülür → Dosya sistemi nesnesi ile diske yazılır → Onay bildirimi gösterilir

Kullanıcı Arayüzü

Kod Editörü

C kaynak kodunun gösterildiği ve hata konumlarına navigasyon yapıldığı editör alanı

Bulgu Listesi

Analiz bulgularını listeleyerek kullanıcıya sunan ve filtreleme imkânı sağlayan panel

Metrik Paneli

Hesaplanan kod metriklerini kullanıcıya özet biçimde gösteren bileşen

Analiz Sonuçları

10

Toplam Satır

Analiz edilen kod satır sayısı

1

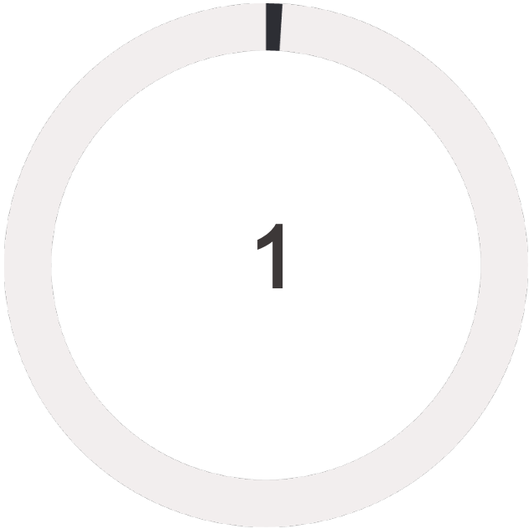
Fonksiyon

Tanımlanan fonksiyon sayısı

0

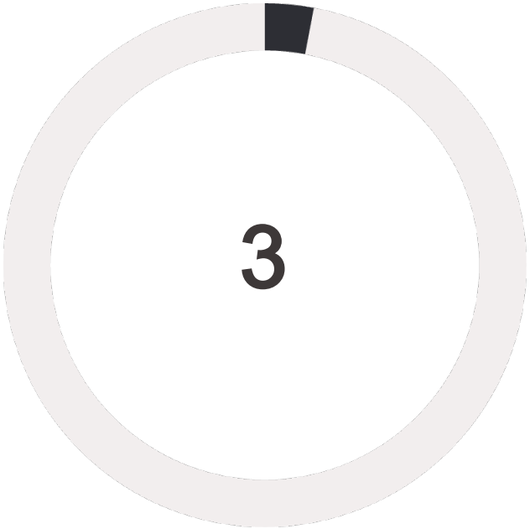
Struct

Tanımlanan yapı sayısı



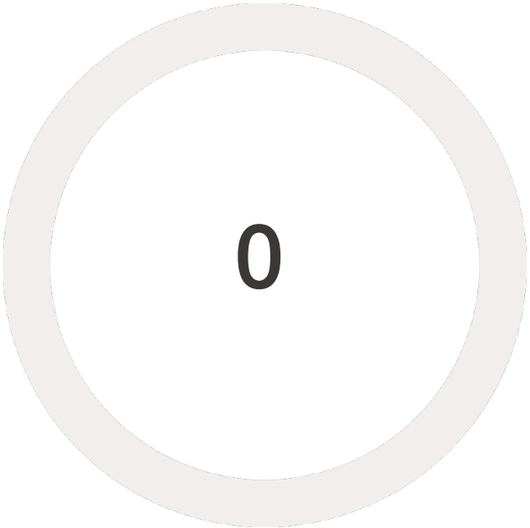
Kritik Hata

Derleme hatasına neden olacak sorunlar



Uyarı

Potansiyel sorunlar ve kötü pratikler



Bilgi

Bilgilendirme amaçlı öneriler

Dinlediğiniz İçin Teşekkürler

Proje Bilgileri

- Ders: BİL 204 / YAZILIM MÜHENDİSLİĞİ
- Bölüm: BİLGİSAYAR MÜHENDİSLİĞİ
- Grup: CodeSentinel
- Tarih: 06.04.2026

Takım Üyeleri

- Ferhat Enes KOCABIYIK
- Hacı Ömer OĞUZ
- Hüseyin Mert KANLIDERE
- Kemal BABAOĞLAN
- Mehmet Enes İYİŞENYÜREK
- Mehmet UÇAR